

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

REDES DE COMPUTADORAS

TRABAJO PRÁCTICO N°5

INFRAESTRUCTURA DE SERVICIOS Y TRÁFICO DE RED

ALUMNOS:

BENAVIDES, MARÍA CANDELA

FARIÑAS, RAFAEL

MELIA, NICOLÁS

SALINAS, JOAQUÍN

DOCENTES:

SANTIAGO MARTIN HENN

FACUNDO NICOLÁS OLIVA CUNEO

CÓRDOBA,

ÍNDICE GENERAL

1. Introducción	4
2. Reconocimiento de Arquitectura	5
2.1. Firewall	5
2.2. Load Balancer (Balanceador de Carga)	5
2.3. Queue (Cola de Mensajes)	6
2.4. Compute (Instancia de Cómputo)	6
2.5. Serverless Function (Función Serverless)	6
2.6. SQL DB (Base de Datos Relacional)	7
2.7. NoSQL (Base de Datos No Relacional)	7
2.8. Cache (Caché)	7
2.9. CDN (Red de Distribución de Contenido)	8
2.10. Storage (Almacenamiento de Objetos)	8
2.11. Search Engine (Motor de Búsqueda)	8
2.12. Réplica (Réplica de Lectura)	9
3. Tipos de Tráfico	10
3.1. Análisis de la interacción entre tipos de tráfico	12
4. Test de Queues (Colas de Mensajes)	13
4.1. Configuración del experimento	13
4.2. Observaciones y análisis	14
4.2.1. Comportamiento al incrementar el rate de tráfico	14
4.2.2. Comportamiento al bajar el rate a cero abruptamente	14
4.2.3. Relación con conceptos de Stallings	14
5. Primera Infraestructura Mínima	15
5.1. Descripción de la arquitectura inicial	15

5.1.1. d) Momento de fallo	16
5.2. Análisis del fallo	16
6. Escalabilidad y Balanceo	18
6.1. Estrategia 1: Agregar más capacidad de cómputo (escalado horizontal)	18
6.1.1. Descripción	18
6.1.2. Análisis	18
6.2. Estrategia 2: Agregar Cache	19
6.2.1. Descripción	19
6.3. Conclusión: ¿Escalar horizontalmente siempre mejora el sistema? . . .	19
7. Sobrevivir (Modo Survival)	20
7.1. Diseño de la arquitectura final	20
7.1.1. Justificación de cada componente	20
7.2. Capturas de pantalla	21
7.3. Análisis del desempeño	21
7.3.1. Primer cuello de botella	21
7.3.2. Componente a escalar con más presupuesto	21
7.3.3. Reflexión sobre la estrategia de diseño	21
8. Conclusiones	22

1. Introducción

El presente trabajo práctico tiene como objetivo comprender cómo una arquitectura de servicios responde ante distintos tipos de tráfico, relacionar componentes de infraestructura *cloud* con conceptos de redes tales como ruteo, balanceo de carga, almacenamiento, bases de datos, caché, colas y filtrado de tráfico malicioso, y analizar fallas, cuellos de botella y decisiones de escalabilidad.

Para ello se utiliza **Server Survival**, un simulador de infraestructura que representa un sistema que recibe tráfico de distintos tipos y debe procesarlo correctamente manteniendo presupuesto, reputación y salud de los servicios. Aunque el entorno es lúdico, los conceptos son reales: un sistema mal diseñado puede fallar por sobrecarga, mala distribución del tráfico, ausencia de caché, falta de filtrado ante ataques o una base de datos saturada.

2. Reconocimiento de Arquitectura

En esta sección se identifican los componentes disponibles en el simulador Server Survival, describiendo la función de cada uno, su ubicación en el modelo TCP/IP y las consecuencias de su ausencia en una arquitectura real.

2.1. FIREWALL

a) Problema que resuelve: El Firewall filtra el tráfico entrante y saliente según reglas de seguridad predefinidas, bloqueando solicitudes maliciosas, ataques de denegación de servicio (DoS/DDoS) y accesos no autorizados antes de que alcancen los servicios internos.

b) Capa(s) del modelo TCP/IP: Opera principalmente en la capa de **Red** (filtrado por IP de origen/destino y puertos), aunque los *firewalls* de inspección profunda también actúan en la capa de **Transporte** (puertos TCP/UDP) y en la capa de **Aplicación** (análisis de contenido HTTP, DNS, etc.).

c) Consecuencias de su ausencia: Sin firewall, el tráfico malicioso llega directamente a los servidores de aplicación, consumiendo recursos de cómputo, agotando las conexiones disponibles y pudiendo comprometer la integridad del sistema. En el simulador, los ataques MALICIOUS saturan rápidamente la infraestructura si no existe un filtro previo.

2.2. LOAD BALANCER (BALANCEADOR DE CARGA)

a) Problema que resuelve: Distribuye el tráfico entrante entre múltiples instancias de cómputo o servicios, evitando que un único nodo se sature mientras otros permanecen ociosos. Mejora la disponibilidad y la tolerancia a fallos.

b) Capa(s) del modelo TCP/IP: Actúa en la capa de **Transporte** (balanceo L4, basado en IP y puerto) y en la capa de **Aplicación** (balanceo L7, basado en URL, headers HTTP, tipo de contenido).

c) Consecuencias de su ausencia: Sin balanceador de carga, una única instancia de cómputo concentra todo el tráfico. Al aumentar el rate de solicitudes, el servidor se satura, incrementa la latencia y eventualmente comienza a rechazar peticiones, causando pérdida de reputación y fallas del servicio.

2.3. QUEUE (COLA DE MENSAJES)

a) **Problema que resuelve:** Desacopla la generación de solicitudes de su procesamiento. Almacena temporalmente las peticiones cuando el ritmo de llegada supera la capacidad de procesamiento, permitiendo que el sistema las atienda de forma ordenada sin pérdida de datos.

b) **Capa(s) del modelo TCP/IP:** Su función principal reside en la capa de **Aplicación**, ya que actúa sobre mensajes o solicitudes a nivel semántico, independientemente del protocolo de transporte subyacente (AMQP, MQTT, HTTP).

c) **Consecuencias de su ausencia:** Sin una cola, los picos de tráfico que superen la capacidad de cómputo generan pérdida directa de solicitudes. El sistema no tiene mecanismo de amortiguación y el componente de cómputo colapsa ante ráfagas de tráfico.

2.4. COMPUTE (INSTANCIA DE CÓMPUTO)

a) **Problema que resuelve:** Es el componente de procesamiento central: ejecuta la lógica de negocio, procesa solicitudes **READ**, **WRITE**, **SEARCH** y coordina el acceso a bases de datos y otros servicios.

b) **Capa(s) del modelo TCP/IP:** Opera en la capa de **Aplicación**, ejecutando servicios web, APIs **REST**, lógica de negocio y consultas a bases de datos.

c) **Consecuencias de su ausencia:** Sin instancias de cómputo, ninguna solicitud puede ser procesada. Es el componente indispensable de cualquier arquitectura de servicios.

2.5. SERVERLESS FUNCTION (FUNCIÓN SERVERLESS)

a) **Problema que resuelve:** Ejecuta código bajo demanda sin necesidad de mantener servidores permanentemente activos. Es adecuada para tareas eventuales, procesamiento de eventos, transformaciones de datos o lógica que no requiere estado persistente.

b) **Capa(s) del modelo TCP/IP:** Capa de **Aplicación**: responde a eventos **HTTP**, mensajes de cola o triggers de almacenamiento, todos en capa de aplicación.

c) **Consecuencias de su ausencia:** Las tareas de procesamiento eventual o de baja frecuencia deben ejecutarse en instancias de cómputo de uso general,

incrementando innecesariamente los costos operativos al mantener servidores activos para carga esporádica.

2.6. SQL DB (BASE DE DATOS RELACIONAL)

a) **Problema que resuelve:** Almacena datos estructurados con soporte para transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). Es adecuada para datos con relaciones bien definidas, donde la integridad es crítica: órdenes, usuarios, registros financieros.

b) **Capa(s) del modelo TCP/IP:** Capa de **Aplicación:** los clientes se conectan mediante protocolos propios de cada motor (MySQL, PostgreSQL) que operan sobre TCP.

c) **Consecuencias de su ausencia:** Sin base de datos relacional, las operaciones de escritura y lectura de datos estructurados no pueden persistirse de forma confiable. En arquitecturas con alta consistencia requerida, su ausencia implica pérdida de datos o inconsistencias.

2.7. NoSQL (BASE DE DATOS NO RELACIONAL)

a) **Problema que resuelve:** Almacena datos no estructurados o semiestructurados (documentos JSON, pares clave-valor, grafos) con alta escalabilidad horizontal y baja latencia de lectura. Es adecuada para datos de sesión, catálogos de productos, perfiles de usuario o métricas.

b) **Capa(s) del modelo TCP/IP:** Capa de **Aplicación:** acceso mediante protocolos propios (MongoDB Wire Protocol, Redis RESP) sobre TCP.

c) **Consecuencias de su ausencia:** Sin NoSQL, los datos de alta variabilidad estructural deben almacenarse en bases relacionales, lo que complica el esquema y reduce el rendimiento en consultas de lectura masiva.

2.8. CACHE (CACHE)

a) **Problema que resuelve:** Almacena en memoria los resultados de consultas frecuentes, reduciendo la cantidad de accesos a la base de datos y disminuyendo drásticamente la latencia de respuesta para solicitudes repetidas.

b) **Capa(s) del modelo TCP/IP:** Capa de **Aplicación:** el caché intercepta solicitudes a nivel de lógica de negocio o de base de datos antes de que lleguen al

motor de almacenamiento persistente.

c) **Consecuencias de su ausencia:** Cada solicitud de lectura impacta directamente en la base de datos. Con tráfico alto de tipo **READ**, la base de datos se convierte en el cuello de botella principal, aumentando la latencia y pudiendo saturarse.

2.9. CDN (RED DE DISTRIBUCIÓN DE CONTENIDO)

a) **Problema que resuelve:** Distribuye contenido estático (imágenes, videos, hojas de estilo, JavaScript) desde servidores geográficamente próximos al usuario final, reduciendo la latencia y descargando el tráfico de los servidores de origen.

b) **Capa(s) del modelo TCP/IP:** Capa de **Aplicación** (sirve contenido HTTP/HTTPS) y beneficia implícitamente las capas de **Red** y **Transporte** al reducir la distancia física entre cliente y servidor.

c) **Consecuencias de su ausencia:** El tráfico estático (**STATIC**) impacta directamente en los servidores de cómputo, desperdiciando recursos de procesamiento en servir archivos que no requieren lógica de negocio. Con alto volumen de tráfico estático, los servidores se saturan rápidamente.

2.10. STORAGE (ALMACENAMIENTO DE OBJETOS)

a) **Problema que resuelve:** Provee almacenamiento persistente de gran capacidad para archivos, imágenes, videos, backups y otros objetos binarios de gran tamaño, separando el almacenamiento de archivos del procesamiento de datos.

b) **Capa(s) del modelo TCP/IP:** Capa de **Aplicación:** accedido mediante APIs HTTP/HTTPS (similar a S3, Azure Blob Storage).

c) **Consecuencias de su ausencia:** Las solicitudes de tipo **UPLOAD** no tienen destino donde persistir los archivos. El sistema pierde la capacidad de almacenar contenido generado por usuarios, logs o respaldos.

2.11. SEARCH ENGINE (MOTOR DE BÚSQUEDA)

a) **Problema que resuelve:** Permite búsquedas de texto completo e indexadas sobre grandes volúmenes de datos, con capacidad para ordenar

resultados por relevancia. Es esencial para funcionalidades de búsqueda que serían muy costosas en una base de datos relacional.

b) Capa(s) del modelo TCP/IP: Capa de **Aplicación**: expone APIs HTTP/REST para realizar consultas y gestionar índices.

c) Consecuencias de su ausencia: Las solicitudes de tipo **SEARCH** deben procesarse mediante consultas directas a la base de datos (**LIKE** o similares), lo que genera carga excesiva y alta latencia, especialmente con grandes volúmenes de datos.

2.12. RÉPLICA (RÉPLICA DE LECTURA)

a) Problema que resuelve: Mantiene una copia sincronizada de la base de datos principal, distribuyendo las operaciones de lectura entre la instancia primaria y las réplicas. Mejora el rendimiento de lectura y provee redundancia ante fallos del nodo principal.

b) Capa(s) del modelo TCP/IP: Capa de **Aplicación**: la replicación ocurre mediante protocolos propios del motor de base de datos (binlog en MySQL, WAL en PostgreSQL).

c) Consecuencias de su ausencia: Con alto tráfico de lectura, la base de datos principal se convierte en cuello de botella. Además, sin réplica, un fallo del nodo de base de datos implica pérdida total del servicio hasta su recuperación.

3. Tipos de Tráfico

El simulador Server Survival trabaja con seis tipos de solicitudes. A continuación se presenta una tabla comparativa de cada tipo de tráfico, con ejemplos reales, componentes recomendados y riesgos asociados a un procesamiento incorrecto.

Tabla 1: *Tipos de tráfico, componentes recomendados y riesgos*

Tipo	Ejemplo real	Componente recomendado	Riesgo si se procesa incorrectamente
STATIC	Imágenes, archivos CSS, JavaScript, fuentes web de una página	CDN o Storage estático	El servidor de aplicación desperdicia ciclos de CPU sirviendo archivos que no requieren lógica de negocio; colapso ante alto volumen
READ	Consulta de perfil de usuario, listado de productos, dashboards en tiempo real	Cache + Réplica de lectura (SQL/NoSQL)	La base de datos principal se satura; alta latencia y degradación del servicio para todos los usuarios
WRITE	Creación de una orden de compra, registro de usuario, envío de formulario	Compute + SQL DB (con soporte ACID) + Queue	Sin cola, los picos de escritura saturan el servidor; sin transacciones, se producen inconsistencias en los datos
UPLOAD	Subida de foto de perfil, envío de archivo adjunto, carga de video	Storage (almacenamiento de objetos) + Queue	Si se procesa en el Compute directamente, el servidor queda bloqueado durante la transferencia de archivos grandes
SEARCH	Búsqueda de productos por nombre, búsqueda de artículos en un blog, autocompletado	Search Engine (motor de indexación)	Consultas LIKE sobre SQL son extremadamente lentas a escala; la base de datos principal se degrada para todos los demás tipos de tráfico
MALICIOUS	Ataques DDoS, inyección SQL, escaneo de puertos, bots de fuerza bruta	Firewall (primer nodo de la arquitectura)	El tráfico malicioso consume recursos de cómputo legítimos; puede comprometer la integridad del sistema y agotar el presupuesto por reparaciones

3.1. ANÁLISIS DE LA INTERACCIÓN ENTRE TIPOS DE TRÁFICO

Es importante notar que en una arquitectura real los tipos de tráfico coexisten simultáneamente. El simulador refleja esta realidad al mezclar porcentajes de cada tipo en cada escenario. El principal desafío de diseño es garantizar que el procesamiento de un tipo de tráfico no degrade el rendimiento de los demás.

Por ejemplo, un alto volumen de solicitudes **SEARCH** sin un motor de búsqueda dedicado impacta directamente sobre la base de datos SQL, aumentando la latencia de las operaciones **READ** y **WRITE**. De forma análoga, tráfico **MALICIOUS** que no es filtrado por un Firewall consume los recursos del Compute que deberían atender solicitudes legítimas.

4. Test de Queues (Colas de Mensajes)

4.1. CONFIGURACIÓN DEL EXPERIMENTO

Para analizar el comportamiento de las colas bajo diferentes condiciones de tráfico, se desplegó en modo *Sandbox* la siguiente arquitectura mínima:

Internet → Firewall → Queue → Compute

Esta configuración aísla el efecto de la cola eliminando otras variables (balanceo de carga, caché, bases de datos), permitiendo observar directamente cómo la cola amortigua las variaciones de tráfico.

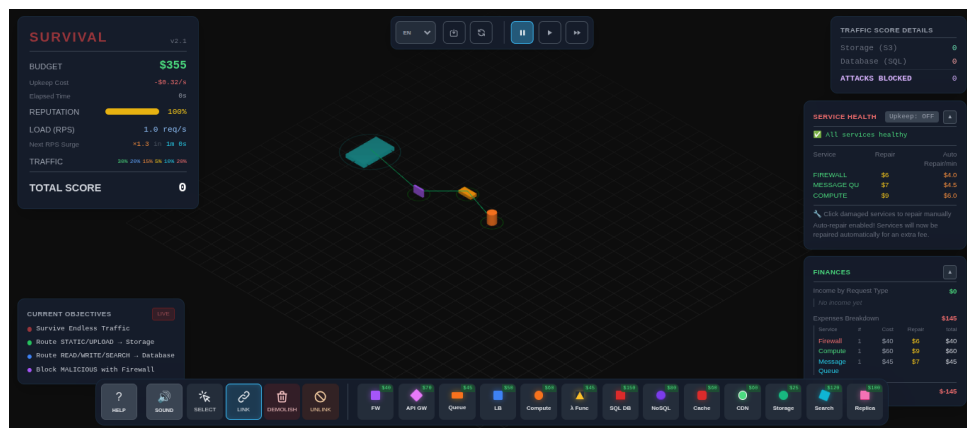


Figura 1: Arquitectura mínima para test de Queue

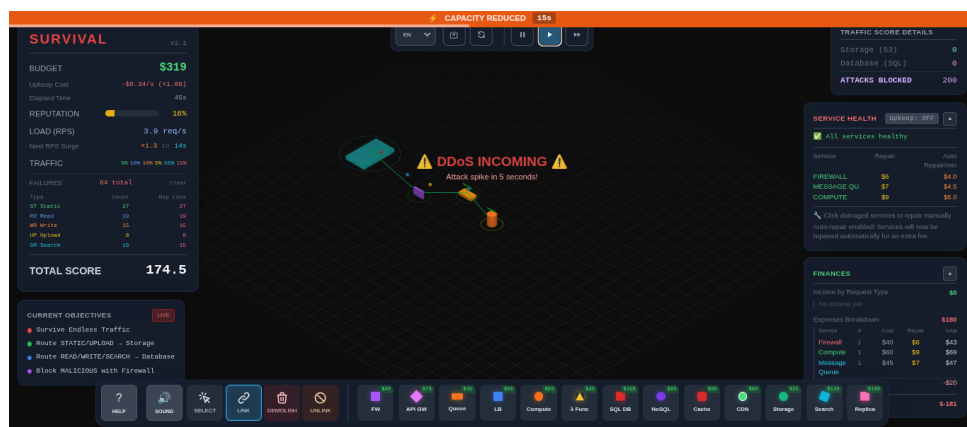


Figura 2: Arquitectura mínima con muchas requests

4.2. OBSERVACIONES Y ANÁLISIS

4.2.1. COMPORTAMIENTO AL INCREMENTAR EL RATE DE TRÁFICO

Al aumentar el rate de solicitudes por encima de la capacidad de procesamiento del Compute, se observa que la Queue comienza a acumular solicitudes. A diferencia de una arquitectura sin cola (donde las solicitudes en exceso se pierden directamente), la Queue actúa como amortiguador: absorbe el excedente de tráfico y lo entrega al Compute a una tasa que este puede manejar.

Este comportamiento es análogo al concepto de **control de flujo** estudiado en la materia: la cola regula la tasa de transferencia entre un emisor rápido (el flujo de tráfico entrante) y un receptor más lento (el Compute).

4.2.2. COMPORTAMIENTO AL BAJAR EL RATE A CERO ABRUPTAMENTE

Al reducir el rate de tráfico a cero rápidamente, la Queue no se vacía de inmediato. El Compute continúa procesando las solicitudes que quedaron encoladas hasta que la cola se vacía completamente. Esto demuestra el efecto de **suavizado**: la Queue introduce un retardo entre el cese del tráfico de entrada y el cese del tráfico procesado por el Compute.

Esta propiedad es útil para absorber ráfagas de tráfico sin pérdida de solicitudes, pero puede introducir latencia adicional en sistemas donde la inmediatez es crítica.

4.2.3. RELACIÓN CON CONCEPTOS DE STALLINGS

El comportamiento de la Queue es directamente comparable al modelo de **flujo de datos y control de congestión** descrito en Stallings (Cap. 11–12). La cola actúa como buffer en la capa de aplicación, de forma análoga a los buffers en la capa de transporte, distribuyendo la carga temporal para evitar pérdidas por desbordamiento.

5. Primera Infraestructura Mínima

5.1. DESCRIPCIÓN DE LA ARQUITECTURA INICIAL

Para atender los cuatro tipos de tráfico requeridos (estático y uploads, lecturas y escrituras, búsquedas, y tráfico malicioso), se diseñó la siguiente arquitectura inicial en modo Sandbox:

Internet → Firewall → Load Balancer → Compute

- ↔ CDN (tráfico STATIC)
- ↔ Storage (tráfico UPLOAD)
- ↔ SQL DB (READ/WRITE)

La elección de cada componente responde a los siguientes criterios:

- **Firewall:** primer punto de contacto, bloquea el tráfico **MALICIOUS** antes de que alcance la infraestructura.
- **Load Balancer:** distribuye el tráfico entre instancias de Compute, evitando sobrecarga en un único servidor.
- **Compute:** procesa las solicitudes de lógica de negocio (**READ**, **WRITE**, coordina con bases de datos).
- **CDN:** atiende el tráfico **STATIC** sin consumir recursos de Compute.
- **Storage:** recibe los archivos de las solicitudes **UPLOAD**.
- **SQL DB:** almacena los datos estructurados para operaciones de lectura y escritura.

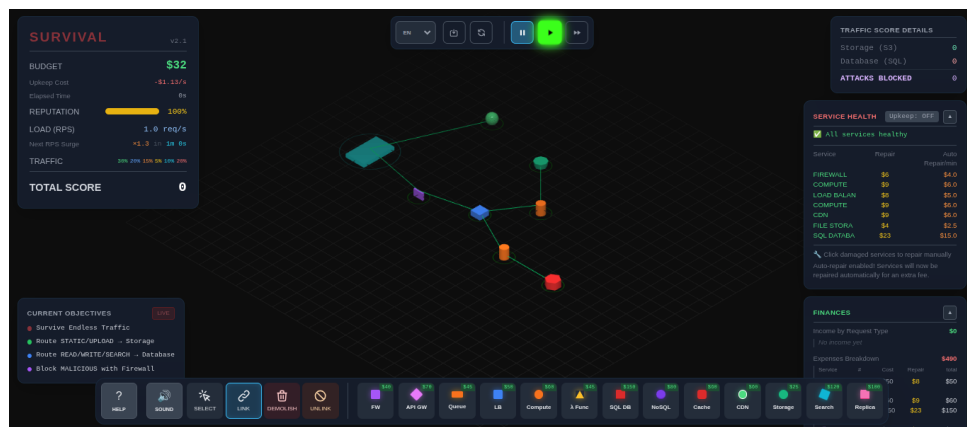


Figura 3: Arquitectura mínima para test de Queue (rate bajo)

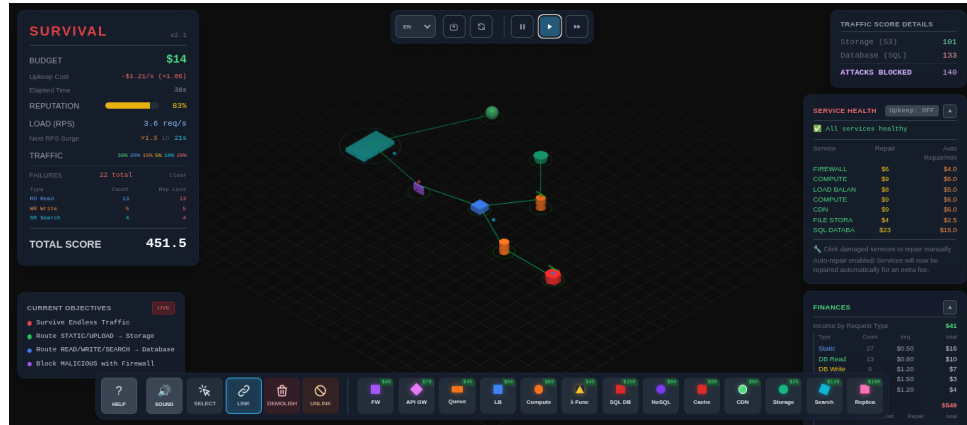


Figura 4: Arquitectura mínima para test de Queue (rate bajo)

5.1.1. D) MOMENTO DE FALLO

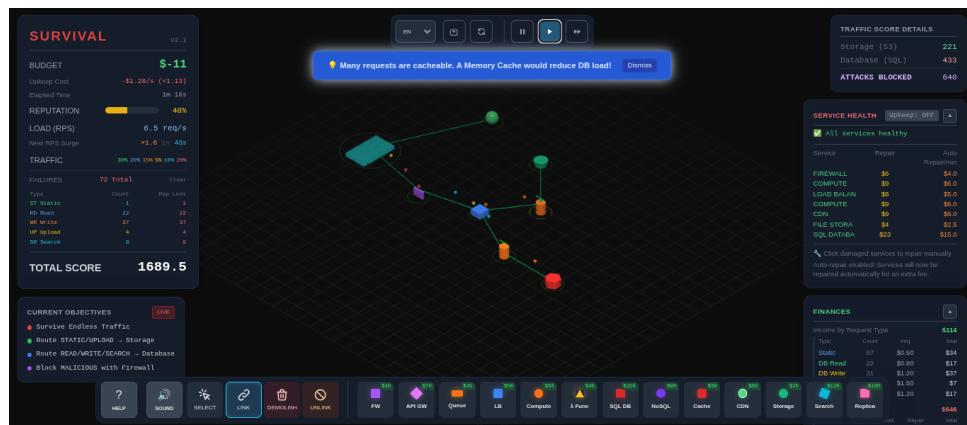


Figura 5: Arquitectura mínima para test de Queue (rate bajo)

5.2. ANÁLISIS DEL FALLO

¿Qué componente falló primero?

[Completar con el componente que falló según la experiencia en el simulador. Ejemplo: la instancia de Compute fue la primera en saturarse al superar el rate los 50 req/s.]

¿Por qué creemos que falló?

[Completar con la justificación observada. Ejemplo: el Compute recibe tráfico

mixto de READ, WRITE y SEARCH simultáneamente. Al aumentar el rate, la cola de solicitudes superó la capacidad de procesamiento del único servidor, incrementando la latencia hasta que comenzó a rechazar solicitudes.]

6. Escalabilidad y Balanceo

En esta sección se evalúan distintas estrategias para mejorar la capacidad de la arquitectura del punto anterior, con el objetivo de soportar mayor tráfico sin degradar la calidad del servicio.

6.1. ESTRATEGIA 1: AGREGAR MÁS CAPACIDAD DE CÓMPUTO (ESCALADO HORIZONTAL)

6.1.1. DESCRIPCIÓN

Se quitó una instancia de Virtual Machine y se agregaron dos instancias instancia de Lambda Function, manteniendo el Load Balancer para distribuir el tráfico entre las tres con menor costo económico.

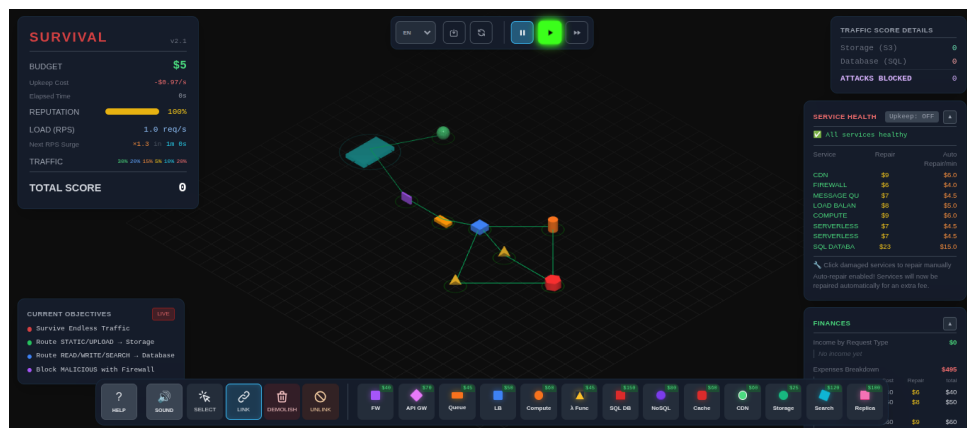


Figura 6: Escalado horizontal: una instancia de VM y dos instancias de Lambdas Functions

6.1.2. ANÁLISIS

El escalado horizontal del Compute desplaza el cuello de botella hacia la base de datos SQL, que ahora recibe mayor cantidad de consultas simultáneas. Esto evidencia que escalar un único componente no garantiza el rendimiento global del sistema: cada capa puede convertirse en el nuevo limitante.

6.2. ESTRATEGIA 2: AGREGAR CACHE

6.2.1. DESCRIPCIÓN

Se incorporó un componente de Cache entre el Compute y la SQL DB, almacenando en memoria los resultados de las consultas de lectura más frecuentes. Esto reduce significativamente la cantidad de consultas que llegan a la base de datos.

6.3. CONCLUSIÓN: ¿ESCALAR HORIZONTALMENTE SIEMPRE MEJORA EL SISTEMA?

No. El escalado horizontal mejora el rendimiento del componente escalado, pero inevitablemente desplaza el cuello de botella hacia el siguiente componente más lento de la cadena. En el simulador se observa claramente que:

- Agregar Compute sin agregar Cache ni réplicas de base de datos traslada la saturación a la SQL DB.
- Agregar Cache sin suficiente Compute no resuelve la saturación de procesamiento.
- La estrategia óptima requiere identificar el cuello de botella actual y escalar el componente correcto en cada momento.

Esto es consistente con la **Ley de Amdahl**: el beneficio del escalado está limitado por la fracción del sistema que no puede ser paralelizada o distribuida.

7. Sobrevivir (Modo Survival)

7.1. DISEÑO DE LA ARQUITECTURA FINAL

Aplicando las lecciones aprendidas en los puntos anteriores, se diseñó una arquitectura más robusta para el modo Survival, orientada a maximizar el puntaje y mantener la reputación en 100 % durante el mayor tiempo posible.

La arquitectura final desplegada es la siguiente:

Internet → Firewall → Load Balancer

→ CDN (tráfico **STATIC**)

→ Compute (x2) → Cache → SQL DB + Réplica

→ Queue → Compute → Storage (tráfico **UPLOAD**)

→ Search Engine (tráfico **SEARCH**)

7.1.1. JUSTIFICACIÓN DE CADA COMPONENTE

- **Firewall**: bloquea el tráfico **MALICIOUS** desde el primer nodo, protegiendo todos los recursos posteriores.
- **Load Balancer**: distribuye el tráfico entre los Computes, evitando saturación de una única instancia.
- **CDN**: descarga completamente el tráfico **STATIC** del Compute, que representaba el mayor porcentaje de solicitudes.
- **Compute (x2)**: dos instancias para procesar **READ** y **WRITE** con capacidad redundante.
- **Cache**: reduce la cantidad de consultas **READ** que alcanzan la SQL DB.
- **SQL DB + Réplica**: la réplica atiende las lecturas, liberando la instancia principal para escrituras.
- **Queue**: amortigua los picos de tráfico **UPLOAD**, evitando bloqueos del Compute durante transferencias de archivos.
- **Storage**: destino final de los archivos cargados.
- **Search Engine**: aísla el procesamiento de **SEARCH** de la base de datos relacional.

7.2. CAPTURAS DE PANTALLA

7.3. ANÁLISIS DEL DESEMPEÑO

7.3.1. PRIMER CUELLO DE BOTELLA

El primer componente en saturarse durante la sesión de Survival fueron las instancias de Compute. A medida que el rate aumentaba, las dos instancias desplegadas alcanzaron su capacidad máxima antes que el resto de los componentes, acumulando solicitudes pendientes y elevando la latencia hasta el punto de comenzar a rechazar peticiones. El Cache mitigó parcialmente la presión al absorber las lecturas más frecuentes, pero no fue suficiente para sostener el volumen mixto de READ y WRITE simultáneo. Una vez saturado el Compute, la degradación se propagó hacia los componentes que dependen de él: las consultas SEARCH comenzaron a fallar al no ser despachadas a tiempo al Search Engine, y parte del tráfico READ desbordó el Cache impactando sobre la SQL DB.

7.3.2. COMPONENTE A ESCALAR CON MÁS PRESUPUESTO

Se agregaría una segunda instancia del Search Engine, ya que las búsquedas constituyeron la principal fuente de fallos durante la sesión de Survival. Alternativamente, agregar más réplicas de lectura de la SQL DB reduciría la latencia de las consultas READ que actualmente desbordan el Cache.]

7.3.3. REFLEXIÓN SOBRE LA ESTRATEGIA DE DISEÑO

La partida de Survival evidenció que no existe una arquitectura universalmente óptima: la configuración óptima depende de la distribución del tráfico en cada momento. El modo Survival introduce cambios progresivos en el rate y en la mezcla de tráfico, forzando adaptaciones dinámicas de la infraestructura.

La clave para alcanzar altos puntajes es mantener un balance entre el costo de la infraestructura y el ingreso por solicitudes procesadas correctamente, optimizando los componentes de mayor impacto antes de que la reputación caiga por debajo de niveles críticos.

8. Conclusiones

El trabajo práctico permitió vincular conceptos teóricos de redes de computadoras con escenarios prácticos de diseño de infraestructura cloud, utilizando el simulador Server Survival como herramienta de experimentación.

Las principales conclusiones obtenidas son:

- **Cada tipo de tráfico requiere un componente especializado:** intentar procesar todo el tráfico a través del Compute general genera cuellos de botella evitables. El CDN para tráfico estático, el Search Engine para búsquedas y el Storage para uploads son especializaciones que mejoran significativamente el rendimiento general del sistema.
- **La seguridad es el primer requisito, no una optimización:** el Firewall debe ser el primer nodo de cualquier arquitectura. El tráfico malicioso que no es filtrado consume recursos legítimos y puede comprometer la integridad del sistema completo.
- **Escalar no siempre implica agregar más del mismo componente:** la estrategia más efectiva es identificar el cuello de botella real en cada momento. Agregar Cache puede ser más efectivo que agregar Compute si el limitante es la base de datos, y agregar réplicas de lectura puede ser más efectivo que Cache si el problema es la capacidad del motor SQL.
- **Las colas de mensajes son fundamentales para sistemas resilientes:** el experimento de Queue demostró que la amortiguación de ráfagas de tráfico mediante buffers en la capa de aplicación es directamente análoga al control de flujo y congestión estudiado en la materia a nivel de transporte, reforzando la aplicabilidad universal de estos conceptos.
- **El costo operativo es una restricción de diseño real:** la decisión de agregar componentes no depende solo del rendimiento técnico sino del presupuesto disponible. Un sistema perfectamente escalado pero inviable económicamente no es una solución válida en entornos reales.

En conjunto, el simulador resultó una herramienta eficaz para experimentar con conceptos de Stallings aplicados a arquitecturas modernas de servicios distribuidos, evidenciando que los principios de control de flujo, encapsulamiento por capas, redundancia y segmentación de tráfico son tan relevantes en la capa de aplicación como en las capas inferiores de la pila TCP/IP.